CSC311: Introduction to Machine Learning

Project Report

What Does a Painting Taste Like?

Predicting Painting Identity from Class Responses

Submitted by

Abigail Xi Chen **Zihan Jin** **Qifan Yang** **Ruoyu Zhou**

University of Toronto

Winter 2026

1 Executive Summary

This project addresses a three-class painting classification task: predicting which of three iconic artworks (*The Persistence of Memory*, *The Starry Night*, or *The Water Lily Pond*) by using up to 14 features, including numerical ratings, Likert scales, free-text responses, price estimates, and multi-select categories. We explored three model families: **Logistic Regression**, a **Multi-Layer Perceptron (MLP)**, and a **Random Forest**. The Random Forest is our best-performing model, achieving a test accuracy of 89.1% and a macro-F1 score of 0.891 on the held-out test set (Cross-Validation (CV) macro-F1 score: 0.882 ± 0.011, training macro-F1 score: 0.9683). It outperformed the other two families primarily because it handles the heterogeneous, mixed-scale feature space, combining dense numeric and ordinal features with high-dimensional sparse TF-IDF text features, without requiring feature scaling, and because ensemble averaging across 100 trees substantially reduces the variance that individual decision trees suffer from on a dataset of this moderate size.

2 Data Exploration

2.1 Dataset Overview

The dataset contains 1,686 observations collected from 562 unique students, each of whom evaluated all three paintings — *The Persistence of Memory*, *The Starry Night*, and *The Water Lily Pond* — yielding perfectly balanced targets across the three classes. Each observation has 14 feature columns spanning five distinct types (Table 1). The target variable is the painting name.

Table 1: Feature types, counts, and pre-processing summary.

Feature type	Features	Count	Pre-processing applied
Numerical	Emotion intensity from 1 to 10 (Q1), colour count (Q7), object count (Q8)	3	Median imputation for missing values; log transform for skewed data
Likert (ordinal)	Sombre / content / calm / uneasy from 1 to 5 (Q3 - Q6)	4	Extract leading digit; mode imputation for missing values
Price (free text)	Willingness-to-pay in CAD (Q9)	1	Parse ranges to midpoint, shorthand (1k→1000); median imputation for missing values
Multi-select categorical	Room choice (Q10), viewing companion (Q11), season (Q12)	3	Normalize separators; one-hot encode; missing → "No Answer"
Free-text	Feeling (Q2), food analogy (Q13), soundtrack (Q14)	3	Lowercase and strip; TF-IDF (max 500 features, 1–2 n-grams); missing → empty string



(a) Mean Likert scores

(b) Seasons per painting

Figure 1: Heatmaps of feature distribution (Q3-Q6, Q12) over classes

Feature distributions reveal clear class-conditional structure. Emotion intensity (Q1) is comparable across paintings (class means 6.08–6.53), contributing limited discriminative power on its own. Colour and object counts (Q7, Q8) show greater between-class variability, especially *The Starry Night*, which has substantially higher variance in object counts. The Likert features (Q3–Q6) are the most discriminative: *The Water Lily Pond* scores highest on *content* and *calm* and lowest on *sombre* and *uneasy*, while *The Persistence of Memory* shows the opposite pattern. The season variable (Q12) maps cleanly to painting identity as Figure 1, whereas room (Q10) and companion (Q11) exhibit 31 unique multi-label combinations each, suggesting lower individual discriminability.

2.2 Data Quality and Issues

Missing values: Every feature column contains missing entries, ranging from 3.9% (emotion intensity) to 5.0% (soundtrack text). Missingness is consistent across the three classes, suggesting it is missing at random rather than class-dependent.

Outliers and inconsistencies: The object-count column contains implausible values (e.g., 100 objects), and willingness-to-pay responses include free-text noise such as "$5", "a", "100,000,000", and shorthand like "1k" and "over a billion". Likert-scale responses were recorded as composite strings requiring digit extraction. These issues confirmed the need for robust, custom preprocessing before modeling.

2.3 Preprocessing

All transformations, as summarized in Table 1, were fitted on the training/validation pool only and applied to the test set without refitting. Median imputation was used for numeric and price columns rather than the mean, as the pronounced right-skew identified above makes the mean unstable; mode imputation was used for Likert columns to preserve integer-valued ordinal structure. Log transformation ($\log(x+1)$) was applied to the skewed count and price columns to compress the long tail and reduce the leverage of extreme values. One-hot encoding was applied to multi-select categorical columns after separator normalization; missing entries were assigned an explicit `No Answer` category rather than dropped, as non-response may itself carry class signal. For TF-IDF vectorization, the three free-text columns were concatenated before fitting to pool vocabulary and stabilize IDF estimates across semantically related columns. The vocabulary was capped at 500 features, preliminary experiments showed negligible macro-F1 gain beyond this threshold, and bigrams were included alongside unigrams to capture phrases that carry stronger class signal than individual tokens.

2.4 Data Splitting

Because each student contributed exactly three rows, these triplets are tightly correlated. Splitting them across folds would allow label information to leak from one painting response to another. To prevent this leakage, splits were performed at the student (`unique_id`) level rather than the row level. Specifically, students were randomly assigned to an 80% training/validation pool and a 20% held-out test set using stratified sampling to preserve the 1:1:1 class ratio. The test set was locked away and not used during any preprocessing, feature engineering, or hyperparameter tuning step. Within the training pool, 5-fold stratified cross-validation was used for model selection.

3 Methodology

3.1 Model Families

Compared to the KNN baseline, we trained and compared three model families on this three-class painting classification task.

Logistic Regression serves as our linear model. It estimates class probabilities via the softmax function and was implemented from scratch in NumPy, minimizing the multiclass cross-entropy loss with batch gradient descent and L2 regularization. Despite being a linear classifier, it is appropriate for our high-dimensional TF-IDF features and provides an interpretable weight vector for each class. Its relatively low capacity makes it a useful lower-bound reference against which the other two families can be compared.

Random Forest is a bagging ensemble of decision trees. Each tree is trained on a bootstrap sample and makes splits by maximizing the reduction in Gini impurity; final predictions are determined by majority vote across all trees. This family suits our dataset because it handles heterogeneous feature types (numeric, ordinal, one-hot, TF-IDF) without requiring feature scaling, captures non-linear interactions and threshold effects without distributional assumptions, and reduces variance through ensemble averaging, particularly valuable given the moderate dataset size of 1,686 rows.

Multi-Layer Perceptron (MLP) is appropriate here because the feature space is high-dimensional and heterogeneous, combining numeric ratings, ordinal Likert responses, TF-IDF text vectors, and one-hot encoded categorical labels, and an MLP can learn non-linear interactions across these diverse signal types without requiring manual feature engineering of cross-terms.

3.2 Optimization and Regularization

Logistic Regression: trained with batch gradient descent on the multiclass cross-entropy loss. L2 regularization (penalty λ) was applied to the weight matrix to limit overfitting to rare tokens. The loss with regularization is:

$$\mathcal{L} = -\sum_{i=1}^{N}\sum_{k=1}^{K} y_{ik}\log(\hat{y}_{ik}) + \lambda\|W\|^2$$

Multiple learning rates and epoch counts were explored via grid search.

Random Forest: does not use gradient-based optimization. Trees are grown by recursively selecting the feature and threshold that maximally reduce Gini impurity. Regularization is achieved by limiting `max_depth` (preventing overly complex trees) and applying cost-complexity pruning via `ccp_alpha` (removing weaker branches to produce simpler, more generalizable trees).

MLP: trained with the Adam optimizer (adaptive moment estimation) for up to 300 epochs. Early stopping with a patience of 10 epochs monitored validation loss to halt training when no improvement was observed, preventing overfitting and unnecessary computation. L2 weight decay (`alpha`) provided additional regularization. No manual learning-rate schedule was applied, as Adam handles effective step-size adjustment internally.

3.3 Validation Method and Hyperparameter Tuning

All three models were tuned using **5-fold stratified cross-validation** on the 80% training pool, split at the `unique_id` level to prevent data leakage between the three correlated responses from each student. In each fold, all preprocessing transformers (TF-IDF vocabulary, imputation medians, one-hot encoder categories) were fitted solely on the four training folds and applied to the held-out fold without refitting. An exhaustive grid search was performed independently for each model family; the configuration with the highest mean macro-F1 across the five folds was selected. Table 2 summarizes the search space and best values found.

Table 2: Hyperparameter search space and best values per model.

Model	Hyperparameter	Values explored	Best value
Logistic Regression	Learning rate	{0.01, 0.005, 0.001}	0.01
	Training epochs	{500, 800, 1000}	1000
	L2 penalty λ	{0, 0.01, 0.1}	0
Random Forest	`n_estimators`	{100, 200, 500}	100
	`max_depth`	{10, 20, None}	20
	`ccp_alpha`	{0.0, 0.0005, 0.001, 0.005, 0.01}	0.001
MLP	Hidden layer sizes	{(50,), (100,), (50,50)}	(100,)
	L2 regularization α	{0.0001, 0.001}	0.0001
	Activation function	{ReLU, Tanh}	ReLU

Each model was tuned independently using stratified k-fold cross-validation, optimizing for macro-F1. Hyperparameter configurations were evaluated across the grid search space, and the best-performing configuration for each model was selected before any cross-model comparison was made.

For **Logistic Regression**, the best configuration identified was $C = 0.01$ and `max_iter` = 1000, yielding a mean CV accuracy of 0.7375 and a mean CV macro-F1 of 0.7234.

For the **Random Forest**, the best configuration used `n_estimators` = 100, `max_depth` = 20, and `ccp_alpha` = 0.001, yielding a mean CV accuracy of 0.8820 and a mean CV macro-F1 of 0.8813.

For the **MLP**, the optimal configuration used two hidden layers of 50 units each (`hidden_layer_sizes` $= (50, 50)$), `alpha` $= 0.001$, and ReLU activation, achieving a mean CV macro-F1 of 0.8665.

3.4 Evaluation Metrics

Across all models, we use multiple evaluation metrics. **Accuracy** measures overall classification correctness and is a natural choice given the perfectly balanced 1:1:1 class distribution; a random baseline would score 33.3%. **Macro-averaged F1-score** assigns equal weight to each of the three painting classes regardless of support, making it sensitive to per-class weaknesses that accuracy can obscure; it served as the primary criterion for model selection during hyperparameter tuning. **Log loss** (MLP only) penalizes confident but incorrect probability estimates, providing a calibration-aware view of model quality beyond hard-label metrics. **Learning curves** (Random Forest) were plotted with training accuracy and CV accuracy against increasing training-set size, to diagnose bias-variance behaviour and confirm that the final model is well-fitted rather than underfitting or memorizing.

3.5 Implementation Details

All models were implemented in **Python**. The Random Forest and MLP used scikit-learn [1] (`RandomForestClassifier`, `MLPClassifier`, `TfidfVectorizer`, `OneHotEncoder`, `StratifiedKFold`), with pandas and NumPy for data manipulation. Logistic Regression was implemented from scratch in NumPy (softmax, one-hot encoding, batch gradient descent), giving the team direct visibility into the optimization process. Custom preprocessing functions (`preprocess_fit` / `preprocess_transform`) encapsulated all transformations and carried fitted parameters from training to inference, ensuring that no test-set information influenced preprocessing at any stage. Trained Random Forest artifacts were serialized with `joblib` for reproducibility.

4 Results

4.1 Model Comparison

Table 3: training and testing metrics for all three models. Best model highlighted in green.

Model	Train acc.	Train F1	Test acc.	Test F1	Log loss
Logistic Regression	0.7329	0.7189	0.6893	0.6772	—
MLP	0.9035	0.9040	0.8643	0.8664	0.3691
Random Forest (final)	**0.9681**	**0.9683**	**0.8905**	**0.8912**	—

Random Forest is the best model (test accuracy **96.81%**, macro-F1 **0.9683**) and is selected as the final prediction model. The MLP ranks second (86.4% / 0.866), while Logistic Regression is weakest (68.1%/0.6772), confirming that linear boundaries are insufficient for this heterogeneous feature space.

4.2 Error Analysis

The Starry Night is the most error-prone class (precision 0.82, recall 0.87): 9 samples are misclassified as *Persistence of Memory* and 6 as *Water Lily Pond*, likely because its inter-

Table 4: Confusion matrix for the final Random Forest on the test set (rows = true label, columns = predicted).

True \ Predicted	Persistence of Memory	Starry Night	Water Lily Pond
The Persistence of Memory	102	10	0
The Starry Night	9	98	6
The Water Lily Pond	1	11	101

mediate emotional tone overlaps with both neighbouring classes. Crucially, there are **zero confusions** between *Persistence of Memory* and *Water Lily Pond*, showing these two classes are well-separated. Logistic Regression showed far heavier misclassification of *Starry Night* (40 samples misclassified as *Water Lily Pond*), further evidencing the need for non-linear models.

4.3 Test Performance Estimate and Justification

Our projected test accuracy for the final Random Forest is **89.1%**. This is supported by: (i) **CV stability**: 5-fold CV accuracy of 88.2%±1.1% shows low sensitivity to data partition; (ii) **CV–test agreement**: the test score falls within one standard deviation of the CV mean, confirming no optimistic bias; (iii) **learning curves**: training and CV accuracy converge as training size grows, indicating a well-fitted model with diminishing variance; and (iv) **metric consistency**: test accuracy (89.1%) and macro-F1 (89.1%) are essentially identical, confirming uniform performance across all three painting classes.

5 Contributions and Learning

- **Abigail Xi Chen:** Trained the Random Forest model and wrote the corresponding methodology section. The biggest lesson learned was that better results usually come from careful choices and testing, not just from making the model more complex.

- **Zihan Jin:** Implemented `predict_all`, wrapped up the methodology section, and completed the final report. Learned how to handle real-world heterogeneous data with mixed feature types, and developed an appreciation for why ensemble methods like Random Forest outperform simpler linear models when features are high-dimensional, sparse, and non-linearly related to the target.

- **Qifan Yang:** Conducted data exploration, led the proposal write-up, and developed the MLP model for the classification task while actively contributing to group discussions on model selection. The most important lesson learned is that building a successful model is not about exhaustively searching all possibilities, but about deeply understanding the data and developing methods accordingly.

- **Ruoyu Zhou:** Trained the Logistic Regression model and wrote the corresponding methodology section. The key lesson learned was the complete process of building a model, from data preprocessing to final training, and overcoming challenges through discussion and continuous experimentation.

References

[1] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[2] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed. Springer, 2009.